

Topic:

Self-Healing Framework — Resilient Locator Engine

Scope:

The Self-Healing Framework is a shared library imported by the Swagger and Jira Playwright suites. When a test's locator fails (UI rebuild, class rename, element-type change, workflow change), it transparently substitutes a working locator at runtime so the test keeps moving — instead of failing and forcing a developer to patch the page object. Every successful heal is persisted to a local SQLite database via `sql.js` (embedded, zero-native-dependency). On the next run, the broken locator is served from cache in microseconds — no LLM call, no page scan. The original page-object selectors are never overwritten on disk; the engine only diverts at runtime. A per-run `healing-summary.json` records healed + failed locators for the orchestrator's healing dashboard.

Key Features of the Self-Healing Framework:

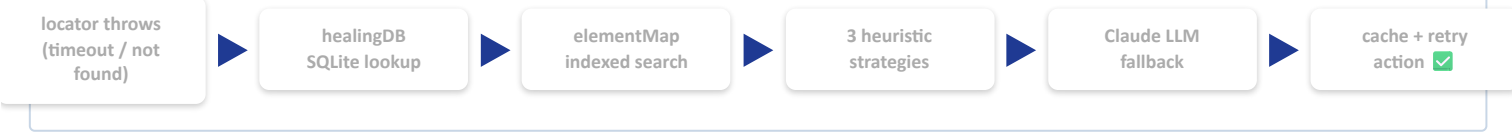
- 6-Step Healing Priority Ladder:** SQLite cache → Element Map → Class-name change → Element-type change → Workflow change → Claude LLM (last resort). First strategy that works wins; result is cached.
- SQLite Persistence (`sql.js`):** `healed_locators` table keyed by original locator — `healed_locator`, `description`, `heal_count`, `first_healed_at`, `last_healed_at`; sub-millisecond lookup; DB file commits with the project so cache survives CI runs.
- Scan-Once Element Map:** First failure scrapes the page ONCE and builds five in-memory indexes (`idIndex`, `classIndex`, `textIndex`, `ariaIndex`, `testIdIndex`). Subsequent failures search the map — no second scrape, no second LLM call.
- Three Heuristic Strategies:** Class-name change (UI rebuild renamed CSS classes), element-type change (`<input>` → `<button>`), workflow change (new step inserted / element under new wrapper) — all resolved without LLM cost.
- Claude LLM Fallback:** Last resort — `claudeHealingService.js` sends broken locator + element description + truncated HTML (default 8000 chars via `HEALING_MAX_HTML_CHARS`) to Claude (Sonnet 4.6), receives one selector with no markdown / commentary.
- Opportunistic, Non-Destructive:** Source page-object selectors never overwritten on disk — engine only diverts at runtime; if original starts working again the cache entry is simply unused.
- Per-Run Healing Report:** `healing-report/healing-summary.json` with `totalHealed`, `totalFailed`, `healed[]/failed[]` (with original locator, healed locator, strategy, page URL).
- Healing Miss Logging:** If all six strategies fail, the engine logs the miss + re-throws the original Playwright error — test fails honestly with full traceability.

Why Use the Self-Healing Framework?

In any real UI test suite, the #1 source of flakiness is locator drift: classes get renamed, wrappers get inserted, designers swap an `<input>` for a `<button>`. Each drift breaks dozens of tests at once. The traditional response is a developer manually patching page objects — slow, expensive, easy to get wrong. Self-healing flips that: the test fixes itself the first time it sees the drift, caches the fix, and never pays for it again. The Swagger and Jira frameworks both depend on this module — which is why it lives at the top level of the repo as a shared library rather than inside any single framework.

Self-Healing Priority Ladder & Architecture

| # | Strategy            | Cost         | Handles                                                                                |
|---|---------------------|--------------|----------------------------------------------------------------------------------------|
| 1 | SQLite cache        | µs           | Any locator healed in a previous run — exact-match broken locator                      |
| 2 | Element Map         | ~1 ms        | Same page, multiple failures — indexed by id / class / text / aria / data-testid       |
| 3 | Class-name change   | ~10 ms       | UI rebuild renamed CSS classes but kept structure                                      |
| 4 | Element-type change | ~10 ms       | Same role, different tag (e.g. <code>input[type=submit]</code> → <code>button</code> ) |
| 5 | Workflow change     | ~50 ms       | Extra step inserted; element under new wrapper / new page                              |
| 6 | Claude LLM          | ~1–3 s + API | Last resort — anything heuristics can't resolve                                        |



Integration with Master Orchestrator, Tooling & CI/CD

Imported directly by the generated Page Objects in Swagger + Jira frameworks via `healLocator(page, brokenLocator, description)`. Built on `Node.js` + `sql.js` (embedded SQLite) + `Anthropic Claude (Sonnet 4.6)` + `Playwright (DOM access)` + `axios` + `dotenv`. Four source files: `selfHealingLocator.js` (orchestrator), `healingDB.js` (SQLite persistence), `elementMap.js` (in-memory index), `claudeHealingService.js` (LLM fallback). Env knobs:

`ANTHROPIC_API_KEY` (only if LLM is reached), `HEALING_LLM_MODEL` , `HEALING_MAX_HTML_CHARS` . At session end, the orchestrator's `healingBridge.js` aggregates per-framework `healing-summary.json` into one consolidated view ( `node index.js --healing-report` ).

**References:** [sql.js.org](#) · [docs.anthropic.com](#) · [playwright.dev](#) · [selenium.dev/documentation](#)

**Courtesy:** K Jyothi Prakash (2445084)

**Contact us @** [kaitepalli.jyothiprakash@cognizant.com](mailto:kaitepalli.jyothiprakash@cognizant.com)

**Visit us @** AI Test Automation Framework — 2026